

# PCMB INTERACTIVE CET MASTER BUILD GUIDE + MASTER PROMPT

A production-ready direction for a complete Physics, Chemistry, Mathematics & Biology learning platform

**Core philosophy:** Learn → Visualize → Practice → Test → Revise

Visual language: minimal, warm, creamy, white, calm and premium — never cluttered or gaming-like.

**IMPORTANT:** This document is the source-of-truth brief for an AI coding agent. It defines the product goal, UX, architecture, interaction philosophy, content system, quality bar and implementation priorities.

# 1. THE MAIN GOAL

**Build a complete interactive MHT-CET learning platform for PCMB students.** The platform must not feel like a PDF reader, generic LMS, coaching portal, or notes website. It should behave like a modern digital learning environment where difficult concepts become understandable through interaction and visualization.

The system must support four academic tracks: Physics, Chemistry, Mathematics and Biology. The architecture must be designed from day one so that new chapters, concepts, simulations, questions and tests can be added without rebuilding the application.

The first implementation focus is not a throwaway prototype. The first chapter should be treated as a real production-quality chapter and should establish reusable systems for the entire platform.

## 2. PRODUCT PRINCIPLES

- **Clarity over decoration.** Every visual element must help the student learn, navigate or make a decision.
- **Interactive before excessive text.** When a concept can be demonstrated with a simulation, graph, diagram or manipulable model, prefer that over a wall of prose.
- **Reusable architecture.** Chapters are content packages that plug into shared learning components.
- **Progressive difficulty.** Move from intuition → concept → formula → worked example → practice → CET-level questions.
- **Fast feedback.** Questions should explain why an answer is correct or incorrect.
- **Mobile-first, desktop-excellent.** The experience must work on phones but use larger screens intelligently.
- **Accessibility and readability are mandatory.** Do not sacrifice legibility for aesthetics.
- **No fake 3D.** Use 3D only where it materially improves understanding.

## 3. VISUAL & UI DIRECTION

### Overall aesthetic

Use a warm, minimal, creamy-white visual system. Think premium study desk, modern textbook, scientific visualization lab and calm productivity software—not gaming UI.

- Primary background: warm ivory / creamy off-white.
- Surfaces: clean white cards with subtle warm-gray borders.
- Text: deep charcoal, not absolute black where possible.
- Accent colors: restrained and purposeful. Subject colors may differentiate Physics/Chemistry/Maths/Biology, but the interface must remain visually unified.
- Borders: thin and subtle.
- Shadows: soft, low-opacity, used sparingly.
- Corner radius: modern but not excessively rounded.
- Typography: highly readable, clean, strong hierarchy.
- Motion: short, smooth transitions. No distracting bouncing, glowing or constant movement.
- Charts/simulations: crisp, high-contrast and scientifically legible.

### Avoid

- Dark/neon dashboard aesthetic.

- Overuse of glassmorphism.
- Huge gradients everywhere.
- Unnecessary 3D objects.
- Decorative animations that slow down learning.
- Tiny text or low-contrast labels.
- Generic stock illustrations.
- Dashboard clutter.

## 4. INFORMATION ARCHITECTURE

The top-level application should include:

- **Dashboard:** continue learning, progress, weak areas, recent chapters and recommended next action.
- **Physics:** chapter library, chapter progress and interactive lessons.
- **Chemistry:** same reusable learning structure.
- **Mathematics:** same structure with stronger support for graphing and step-by-step problem solving.
- **Biology:** same structure with diagrams, anatomy, cell and process visualizations.
- **Practice:** chapter/topic-wise question bank.
- **Tests:** timed CET-style tests and mixed-subject tests.
- **Revision:** formulas, key facts, mistakes and saved concepts.
- **Progress:** chapter completion, practice accuracy, test performance and weak-topic analysis.

## 5. CHAPTER EXPERIENCE

Every chapter should follow a consistent learning journey:

**Chapter overview** → **Concept map** → **Interactive lesson** → **Worked examples** → **Guided practice** → **CET MCQs** → **Chapter test** → **Revision summary**

### Chapter overview

- Chapter title and subject.
- Estimated learning time.
- Prerequisites.
- Learning objectives.
- Progress indicator.
- Start/continue button.

### Concept lesson

Each concept should have a concise explanation, an interactive visual where appropriate, a formula/definition panel, a worked example and a short check-for-understanding question.

### End of chapter

Show mastery, accuracy, weak concepts, questions to revisit, and a clear next step.

## 6. INTERACTIVE LEARNING SYSTEM

### Physics

Physics should emphasize motion, vectors, forces, graphs, physical relationships and simulations.

**Initial production chapter: Kinematics.**

- Position and reference point visualization.
- Distance vs displacement using an interactive number line.
- Velocity with controllable motion.
- Acceleration with live parameter changes.
- s-t, v-t and a-t graph visualization.
- Projectile motion with adjustable initial velocity and angle.
- Live values for relevant quantities.
- Formula panel connected to the simulation.
- Numerical examples where changing an input updates the result.

### Chemistry

- Mole/particle/mass relationships.
- Atomic structure visualization.
- Periodic trends.
- Chemical bonding and molecular geometry.
- Thermodynamics and equilibrium visual explanations.
- Reaction/process diagrams where useful.

### Mathematics

- Interactive graphs.
- Sliders for variables and parameters.
- Dynamic geometry.
- Step-by-step equation solving.
- Function transformations.
- Trigonometric visualization.
- Coordinate geometry.

### Biology

- Clickable diagrams.
- Cell structures and organelles.
- Cell division sequence.
- Human physiology processes.
- Plant/animal structure visualizations.
- Process animations with labels and checkpoints.

## 7. REUSABLE COMPONENT SYSTEM

Build a component library before duplicating chapter UI.

- SubjectCard
- ChapterCard
- ConceptCard
- ConceptProgress
- InteractiveCanvas
- SimulationControls
- GraphPanel
- FormulaCard
- DefinitionCard
- WorkedExample
- StepByStepSolution
- MCQCard
- NumericalQuestion
- AnswerFeedback
- HintPanel
- ChapterTest
- Timer
- ProgressRing
- MistakeReview
- RevisionCard
- Search
- Breadcrumbs
- Sidebar/SubjectNavigation

Components must accept structured data rather than hard-coded chapter-specific content wherever possible.

## 8. CONTENT MODEL

Design the content system so chapters can be authored as structured data.

**Suggested hierarchy:** Subject → Chapter → Module → Concept → Explanation/Visual → Example → Question → Test.

A concept object should be capable of storing: title, summary, prerequisites, explanation blocks, formulas, visual type, simulation parameters, examples, questions, difficulty and revision notes.

Question objects should support: question text, options, correct answer, explanation, difficulty, topic, concept ID, question type and optional numerical data.

## 9. PRACTICE & CET ENGINE

- Chapter-wise practice.
- Topic-wise practice.
- Difficulty filters.
- MCQ mode.
- Numerical mode.
- Timed mode.
- Instant explanation.
- Retry incorrect questions.
- Bookmark question.
- Mistake notebook.
- Performance by concept.
- Chapter mastery score.

Do not make the test engine a decorative page. It must be a real reusable system that can later support full-length CET-style testing.

## 10. PROGRESS & PERSONALIZATION

The dashboard should answer three questions immediately: **What did I study? What am I weak at? What should I do next?**

- Overall subject progress.
- Chapter completion.
- Practice accuracy.
- Questions attempted.
- Incorrect-question count.
- Weak concepts.
- Recently studied concepts.
- Revision queue.
- Test scores.
- Streaks only if they genuinely help; avoid manipulative gamification.

## 11. RESPONSIVE UX

Desktop should use the available space for simulations, split views and larger diagrams. Mobile should collapse navigation and prioritize one learning action at a time.

- No horizontal scrolling.
- Touch-friendly controls.
- Simulation controls usable with fingers.
- Graphs remain readable on small screens.
- Sticky progress/navigation only when useful.
- Avoid hover-only interactions.

## 12. TECHNICAL DIRECTION

Use a modern component-based web architecture. The implementation should prioritize maintainability, performance and clean separation between UI, content and simulation logic.

- Frontend: React/TypeScript-style component architecture.
- Styling: utility-first or well-structured CSS system with design tokens.
- Interactive graphics: SVG/Canvas/WebGL/Three.js only where appropriate.
- Animations: a lightweight motion system; avoid animation dependencies unless useful.
- State: centralized only where necessary; local component state for isolated interactions.
- Content: structured data/content modules rather than hard-coded JSX pages.
- Persistence: design a clean abstraction so local storage can work initially and a backend can be added later.
- Performance: lazy-load heavy simulations and avoid loading every chapter asset at startup.

## 13. DESIGN TOKENS

Create centralized tokens for:

- Backgrounds
- Surface colors
- Text hierarchy
- Borders
- Subject accents
- Spacing
- Radius
- Typography sizes
- Motion durations
- Simulation colors and graph styles

The product should look like one coherent platform even when content comes from four different subjects.

## 14. QUALITY BAR

The application should feel finished, not generated.

- No placeholder lorem ipsum.
- No dead buttons.
- No broken links/routes.
- No fake progress numbers.
- No empty cards pretending to be features.
- No console errors in normal use.
- No unnecessary loading screens.
- All major interactions must have clear feedback.
- Animations must not interfere with studying.

- Every chapter screen must be usable with keyboard and touch where relevant.

## 15. DEVELOPMENT STRATEGY

Do not create four disconnected mini-apps. Build one platform with one design system and one learning engine.

**Step 1:** establish app shell, routing, theme, responsive layout and component system.

**Step 2:** establish chapter/content schema.

**Step 3:** implement the complete Kinematics chapter using production-quality reusable components.

**Step 4:** implement practice, question feedback, test and progress systems.

**Step 5:** generalize the content pipeline so additional chapters can be added as data/content modules.

**Step 6:** populate the rest of PCMB progressively.

Important: Step 3 is not a throwaway prototype. It is the first real chapter and should establish the quality standard for the entire product.

## 16. MASTER BUILD PROMPT

Paste the following prompt into the coding agent as the primary product instruction. The agent should treat this document as the source of truth.

You are the lead product architect, senior frontend engineer, interaction designer and educational-technology engineer for a premium interactive MHT-CET PCMB learning platform.

### MISSION:

Build a complete, scalable learning platform for Physics, Chemistry, Mathematics and Biology. This is not a notes website, generic LMS or temporary prototype. Build the real product architecture from the beginning.

### CORE LEARNING LOOP:

Learn → Visualize → Understand → Practice → Test → Review → Revise.

### VISUAL DIRECTION:

Use a minimal warm creamy/ivory-white interface. The product should feel calm, premium, academic and modern. Use deep charcoal typography, clean white surfaces, subtle warm-gray borders, soft shadows, restrained subject accents and excellent spacing. Avoid dark/neon gaming aesthetics, excessive gradients, excessive glassmorphism and decorative clutter.

### PRODUCT STRUCTURE:

Dashboard  
Physics  
Chemistry  
Mathematics  
Biology  
Practice  
Tests  
Revision  
Progress

### ARCHITECTURE:

Create a reusable subject → chapter → module → concept → activity/question architecture. Do not hard-code every chapter as a separate custom page. Build reusable components and structured content schemas.

### REUSABLE SYSTEMS:

ChapterCard, ConceptCard, Progress, InteractiveCanvas, SimulationControls, GraphPanel, FormulaCard, WorkedExample, StepByStepSolution, MCQCard, NumericalQuestion, Feedback, HintPanel, TestEngine, Timer, MistakeReview, RevisionCard and navigation components.

### FIRST PRODUCTION CONTENT:

Build Physics → Kinematics as the first complete production chapter.

#### KINEMATICS MUST INCLUDE:

1. Position and reference point.
2. Distance vs displacement interactive visualization.
3. Velocity simulation.
4. Acceleration simulation.
5. s-t graph.
6. v-t graph.
7. a-t graph.
8. Projectile motion with adjustable angle and initial velocity.
9. Live numerical values.
10. Formula explanations connected to the visualization.
11. Worked examples.
12. Practice MCQs.
13. Numerical questions.
14. Chapter test.
15. Mistake/revision flow.

#### INTERACTION RULE:

Every interactive element must teach something. Do not add 3D merely because it looks impressive. Use 2D/SVG/canvas for graphs and diagrams where it is clearer. Use WebGL/Three.js only when 3D materially improves understanding.

#### CONTENT SYSTEM:

Make chapter content data-driven. A concept should support title, summary, prerequisites, explanation blocks, formulas, visual type, parameters, examples, questions, difficulty, tags and revision notes. Questions should support options, correct answer, explanation, difficulty, concept/topic ID and numerical data where applicable.

#### PRACTICE:

Support chapter practice, topic practice, difficulty, timed mode, instant explanations, retry incorrect questions, bookmarks and mistake review.

#### PROGRESS:

Track studied concepts, chapter completion, practice accuracy, attempted questions, incorrect questions, weak concepts, revision queue and test scores. The dashboard must clearly answer:

- What have I studied?
- What am I weak at?
- What should I study next?

#### RESPONSIVE DESIGN:

Mobile-first but excellent on desktop. No horizontal scrolling. Touch-friendly controls. Graphs and simulations must remain readable. Avoid hover-only interactions.

#### ENGINEERING:

Use a clean React/TypeScript-style component architecture. Keep UI, content and simulation logic separated. Use centralized design tokens. Lazy-load heavy simulations. Avoid unnecessary dependencies. Keep the application fast and maintainable.

#### QUALITY:

No lorem ipsum. No dead buttons. No fake functionality. No broken routes. No placeholder cards pretending to be features. No console errors in normal use. Provide clear feedback for all important actions. Keep animations short and purposeful.

#### IMPORTANT EXECUTION RULE:

Do not stop after creating a visual shell. Implement real interactions and real learning functionality. Do not create a disposable prototype. Build reusable infrastructure first and then implement Kinematics as the first complete chapter.

After Kinematics is working, ensure the architecture can accept Chemistry, Mathematics and Biology chapters without redesigning the platform.

Before considering the build complete, inspect every major route and interaction for responsiveness, usability, consistency, performance and obvious bugs. Fix issues instead of merely reporting them.

#### FINAL PRODUCT FEEL:

A student should feel like they are entering a premium interactive science learning environment—not opening another coaching PDF.

## 17. DEFINITION OF DONE

- The app has a coherent dashboard and subject navigation.
- The creamy minimal design system is consistent everywhere.
- Kinematics is a complete, usable production chapter.
- At least the major simulations are genuinely interactive.
- Practice and test flows work end-to-end.
- Progress and mistake review work end-to-end.
- Content is data-driven and reusable.
- The app is responsive on mobile and desktop.
- The architecture is ready for all four subjects.
- No major placeholder functionality remains in the implemented scope.

## 18. LONG-TERM ROADMAP

**Stage A — Platform foundation:** app shell, design system, routing, content schema, reusable learning components.

**Stage B — Physics:** expand mechanics, waves, thermodynamics, electricity, optics and other relevant chapters according to the official syllabus.

**Stage C — Chemistry:** physical, inorganic and organic learning modules with appropriate visualizations.

**Stage D — Mathematics:** graphing, geometry, algebra, calculus and problem-solving interactions.

**Stage E — Biology:** cell biology, plant biology, animal biology, human physiology and process visualizations.

**Stage F — CET engine:** large question bank, timed mocks, mixed tests, analytics and revision intelligence.

**Stage G — Personal learning:** smarter recommendations based on performance and revision history.

## 19. FINAL PRODUCT NORTH STAR

The platform succeeds when a student can open a difficult chapter, interact with the concept, understand the relationship visually, solve a guided example, practice independently, take a CET-style test, see exactly where they went wrong, and know what to revise next—without leaving the platform.

**North-star sentence:** “Turn difficult PCMB concepts into things students can see, manipulate, understand and master.”